Deadline-Aware Online Optimization for Mobile Edge Computing via Age-Layered Queues

Xin Gao*, Hongwei Shi†*, Chen Su*, Zhi Zheng* and Tao Huang‡*
*Future Network Center, Purple Mountain Laboratories, Nanjing 211111, China
†School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China
‡State Key Laboratory of Networking and Switching Technology,
Beijing Univ. of Posts and Telecom. (BUPT), Beijing 100876, China
Emails: {gaoxin, shihongwei, suchen, zhengzhi, huangt}@pmlabs.com.cn

***Abstract*—Mobile edge computing (MEC) is expected to support mission-critical 6G services with stringent end-to-end delay requirements. However, MEC systems are inherently stochastic and tightly coupled across communication and computation, as task arrivals, wireless channels, and computing resources all vary over time. This makes hard delay provisioning fundamentally more challenging than conventional average-delay optimization. This paper investigates online resource allocation for MEC networks under hard end-to-end delay constraints. We develop an age-layered queueing model that explicitly captures workload urgency in both local and edge queues, and adopt an age-priority service discipline to enable deadline-aware processing. Based on this model, we formulate a long-term stochastic optimization problem to minimize the average system cost, including communication-computation energy consumption and deadline-violation penalties. We then propose an age-layered deterministic-delay optimization (ADDO) algorithm based on Lyapunov drift-plus-penalty optimization, which makes per-slot decisions on local computing, task offloading, and edge CPU allocation without requiring prior knowledge of traffic-arrival or channel statistics. Theoretical analysis establishes an $O(1/V)$ cost-optimality gap with an $O(V)$ backlog tradeoff, where $V > 0$ is the Lyapunov control parameter. Numerical results show that, under appropriate choices of V, ADDO achieves the lowest deadline-violation ratio among all benchmark policies and can completely eliminate deadline violations in certain scenarios.**

***Index Terms*—edge computing, deterministic delay, online optimization, Lyapunov optimization, resource allocation.**

I. Introduction

With the rapid development of artificial intelligence, the Internet of Things (IoT), and next-generation mobile communications, future 6G networks are expected to support a wide range of mission-critical services, such as autonomous driving, industrial automation, and immersive interaction [1]. These applications require not only high data rates and ubiquitous connectivity, but also stringent and verifiable end-to-end delay guarantees. As a result, deterministic latency provisioning has emerged as a fundamental requirement for next-generation wireless architectures.

Mobile edge computing (MEC) is widely recognized as a pivotal paradigm for enabling such latency-sensitive services. By migrating computation and storage resources from centralized clouds to edge nodes in close proximity to end users [2], MEC can significantly reduce service latency, alleviate backbone traffic, and enhance system responsiveness. However, to fully support mission-critical applications, MEC systems must guarantee that each workload is completed within a prescribed deadline, making deterministic delay provisioning a central design objective.

Despite its importance, existing studies only partially address this challenge. On the communication side, deterministic networking technologies such as Time-Sensitive Networking (TSN) and Deterministic Networking (DetNet) mainly focus on predictable transmission latency, while often overlooking the communication–computation coupling inherent in MEC systems [3]. On the MEC side, a substantial body of work on resource allocation targets average-delay minimization, probabilistic delay guarantees, or robust design under uncertainty [4]–[6]. Although these studies provide useful insights, they do not explicitly enforce hard end-to-end deadlines in stochastic MEC environments. In addition, while Lyapunov-based online optimization has been widely adopted for dynamic MEC control due to its independence from prior statistical knowledge [7], [8], existing frameworks are primarily developed for time-average performance metrics and are not directly tailored to deterministic deadline enforcement.

To bridge this gap, this paper investigates online resource allocation in MEC networks subject to hard end-to-end delay constraints. The key challenge lies in how to explicitly represent workload urgency in a stochastic computation–communication coupled system, while preserving the online and distribution-free advantages of Lyapunov optimization. To this end, we develop an age-layered queueing model that partitions both local and edge backlogs according to workload age. Under this model, any workload that reaches its maximum admissible age without being completed is explicitly identified as overdue. We further introduce an age-priority service discipline, under which older workloads are always served first, so that the resulting control policy is inherently deadline-aware.

Based on the above model, we formulate a long-term stochastic optimization problem to minimize the average system cost, including local computing energy, transmission energy, edge computing energy, and deadline-violation penalties, subject to queue stability and resource constraints. We then propose an age-layered deterministic-delay optimization (ADDO) algorithm based on Lyapunov drift-plus-penalty optimization. The proposed algorithm jointly determines local

computing, task offloading, and edge CPU allocation in an online manner without requiring prior knowledge of traffic-arrival or channel-state statistics.

Compared with existing studies, the proposed framework has three main distinguishing features. First, instead of focusing on average delay, probabilistic reliability, or virtual-queue-based surrogate delay metrics, it directly targets hard end-to-end delay constraints at the workload level. Second, by introducing an age-layered queueing model together with an age-priority service discipline, the proposed method makes deadline urgency explicitly visible to the controller in both local and edge queues. This allows the controller to identify critical workloads that are close to deadline violation and to intervene earlier through local computing, offloading, and edge CPU allocation, thereby enabling more effective deadline protection than indirect average-based regulation. Third, the resulting algorithm operates online without requiring prior statistical knowledge, while still admitting provable performance guarantees on the cost–backlog tradeoff. In this way, the proposed framework bridges deterministic delay control and stochastic online optimization in computation–communication coupled MEC systems.

The main contributions of this paper are summarized as follows:

- We develop an age-layered queueing framework for MEC systems with hard end-to-end delay constraints. By explicitly tracking workload age in both local and edge queues and combining it with an age-priority service discipline, the proposed model establishes a tractable connection between deterministic delay requirements and queue evolution.
- We formulate a long-term stochastic optimization problem that jointly captures communication cost, computation cost, and deadline-violation penalties under stochastic arrivals and time-varying wireless channels. This formulation explicitly characterizes the tradeoff between energy expenditure and deadline protection.
- We propose the ADDO algorithm based on Lyapunov drift-plus-penalty optimization. The algorithm makes per-slot decisions on local computing, task offloading, and edge CPU allocation in an online and distribution-free manner, with closed-form local control and efficient bisection-based edge resource allocation.
- We establish theoretical performance guarantees for the proposed algorithm, showing an $O(1/V)$ cost-optimality gap and an $O(V)$ average-backlog tradeoff, where $V > 0$ is the Lyapunov control parameter. Numerical results further show that ADDO achieves the lowest deadline-violation ratio among benchmark policies and can completely eliminate deadline violations in certain scenarios.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model. Section IV formulates the long-term optimization problem. Section V develops the proposed ADDO algorithm and its theoretical analysis. The numerical results are presented in Section VI. Finally, Section VII concludes the paper.

II. Related Work

A. Deterministic Delay Guarantees

Deterministic communication has been widely investigated for latency-critical services. In wired networks, Time-Sensitive Networking (TSN) provides bounded-latency communication through time synchronization and gate control scheduling [9], while Deterministic Networking (DetNet) extends deterministic forwarding and resource reservation mechanisms to IP networks [10]. In wireless systems, ultra-reliable low-latency communication (URLLC) has motivated extensive studies on delay–reliability tradeoffs based on finite blocklength coding, effective capacity, and stochastic network calculus [11], [12].

These works provide important foundations for predictable transmission latency. However, they mainly focus on the communication layer and do not explicitly address end-to-end deterministic latency in MEC systems, where wireless transmission, task offloading, queueing, and edge execution are tightly coupled.

B. Latency Optimization and Online Control in MEC

Latency optimization has been a major topic in MEC research. Existing studies have investigated joint task offloading and resource allocation to reduce delay and improve energy–delay tradeoffs. Representative approaches include risk-aware formulations based on chance constraints or CVaR [13], [14], queueing-theoretic delay analysis [15], and robust optimization under uncertainty [5]. These methods provide useful tools for latency-aware MEC design, but they mainly target average delay performance, probabilistic guarantees, or robustness under prescribed uncertainty models, rather than hard per-task deadline enforcement.

To address time-varying arrivals, channel conditions, and computing demands, prior work has also applied stochastic optimization, Markov decision processes, reinforcement learning, and Lyapunov-based stochastic network optimization to dynamic MEC resource management [16]–[20]. Among them, Lyapunov-based methods are particularly attractive because they enable online control without requiring prior statistical knowledge. Nevertheless, existing online MEC frameworks still mainly optimize time-average metrics such as average delay or long-term cost. Even when virtual queues are introduced, the resulting guarantees are typically expressed in terms of average violations or probabilistic performance, rather than deterministic end-to-end delay guarantees for individual tasks.

In summary, existing studies on deterministic communication mainly address predictable transmission latency, while most MEC resource-allocation methods focus on average-delay optimization, probabilistic delay guarantees, robust design under uncertainty, or distributed coordination with delay guarantees. In particular, URLLC-oriented MEC schemes typically characterize latency reliability through probabilistic queue-length constraints or extreme-event statistics, whereas

Lyapunov-based MEC approaches usually enforce deadline-related requirements through virtual queues and optimize long-term average performance. Different from these lines of work, our method makes deadline urgency directly observable to the controller by tracking workload age in both local and edge queues and prioritizing older workloads in service. This provides a more fine-grained view of deadline risk than conventional backlog-based or virtual-queue-based formulations, which mainly regulate delay indirectly through average quantities. As a result, the controller can distinguish between workloads that are merely accumulated and those that are close to deadline violation, and can therefore intervene earlier through local computing, offloading, and edge CPU allocation. Such a mechanism is particularly advantageous under hard delay constraints, since it enables more timely protection of critical workloads and leads to lower deadline-violation ratios in stochastic MEC systems, while still preserving the distribution-free and provably efficient nature of Lyapunov-based online optimization.

III. System Model

We consider a slotted dynamic mobile edge computing (MEC) network consisting of one edge server[1] and a set of users $\mathcal{K} \triangleq \{1, \dots, K\}$. Time is divided into equal-length slots indexed by $t \in \{0, 1, 2, \dots\}$, where each slot has duration δ. The edge server is co-located with an access point, and all users can either process their workloads locally or offload them to the edge server for remote execution.

A. Workload and Resource Model

For each user $i \in \mathcal{K}$, let $A_i(t)$ denote the workload arriving at the beginning of time slot t, measured in bits.[2] The arrival process $\{A_i(t)\}$ is stochastic and time-varying, and is assumed to be bounded as $0 \le A_i(t) \le A_{i,\max},\ \forall t$. All newly arrived workload is first buffered and then scheduled for processing either locally or at the edge server.

Each workload generated by user i is associated with a *hard end-to-end latency deadline* $D_i \in \mathbb{Z}_+$, meaning that every workload unit must be completed within D_i time slots after its arrival.[3]

Each workload bit of user i requires c_i CPU cycles for execution. User i is equipped with a local processor, whose CPU frequency in slot t is denoted by $f_i^{\mathrm{L}}(t)$. Meanwhile, the edge server allocates a CPU frequency $f_i^{\mathrm{E}}(t)$ to process the offloaded workload of user i. Accordingly, the maximum amount of workload that can be processed within slot t (in bits) by local and edge computing, respectively, is given by

$$\mu_i^{\mathrm{L}}(t) = \delta f_i^{\mathrm{L}}(t)/c_i, \tag{1}$$

$$\mu_i^{\mathrm{E}}(t) = \delta f_i^{\mathrm{E}}(t)/c_i, \tag{2}$$

where δ denotes the duration of one time slot. These service rates characterize the maximum executable workload under the allocated CPU resources.

The local CPU frequency of each user is constrained by the hardware capability as

$$0 \le f_i^{\mathrm{L}}(t) \le f_{i,\max}^{\mathrm{L}}, \quad \forall i, t, \tag{3}$$

while the edge server has a total CPU frequency budget shared among all users, given by

$$\sum_{i \in \mathcal{K}} f_i^{\mathrm{E}}(t) \le f_{\max}^{\mathrm{E}}, \quad f_i^{\mathrm{E}}(t) \ge 0, \quad \forall i, t, \tag{4}$$

where $f_{\max}^{\mathrm{E}}$ represents the total available computational capacity at the edge server in each slot.

Each user may offload part of its local workload over the wireless uplink. Let $p_i(t)$ denote the transmit power of user i in slot t, which subject to

$$0 \le p_i(t) \le p_{i,\max}, \quad \forall i, t. \tag{5}$$

Then the amount of workload that can be offloaded in slot t is modeled as

$$\mu_i^{\mathrm{T}}(t) = \delta B_i \log_2 \left(1 + p_i(t) H_i(t)/(N_0 B_i)\right), \tag{6}$$

where $H_i(t)$ denotes the corresponding channel power gain, B_i is the allocated bandwidth, and N_0 is the noise power spectral density. To capture both large-scale attenuation and small-scale fading, $H_i(t)$ is modeled as

$$H_i(t) = \beta_0 \left(\frac{d_0}{d_i(t)}\right)^{\alpha} \xi_i(t), \tag{7}$$

where $d_i(t)$ is the distance from user i to the edge node, $d_0 = 1$ m is the reference distance, β_0 denotes the channel power gain at the reference distance, and α is the path-loss exponent. The small-scale fading term is denoted by $\xi_i(t) = |g_i(t)|^2$, where $g_i(t) \sim \mathcal{CN}(0, 1)$. Hence, $\xi_i(t)$ follows an exponential distribution with unit mean. We assume a frequency-flat block fading channel, i.e., $H_i(t)$ remains constant within each slot and changes independently across different slots.

B. Age-Layered Queue Model

Each user maintains one *local queue* to buffer newly arrived workloads. At the edge side, the edge server maintains K *logical edge queues*, one for each user to preserve user-specific queue states and allow fine-grained edge CPU scheduling across users under a shared computation budget.

Let $Q_i^{\mathrm{L}}(t)$ and $Q_i^{\mathrm{E}}(t)$ denote the total local and edge backlog of user i at the beginning of slot t, respectively. Newly generated workload first enters $Q_i^{\mathrm{L}}(t)$. From the local queue, workload can either be executed locally or offloaded to the corresponding logical edge queue. The workload offloaded in slot t is assumed to arrive at the edge server at the beginning of slot $t+1$, which captures a one-slot transmission latency and is consistent with the slotted decision model. The edge server then processes the received workload and returns the result to the user. Since the size of the computation result is typically

[1] The proposed framework can be extended to multi-server MEC systems by augmenting the offloading action space and server-side queue states.

[2] Equivalently, the workload can be measured in packets or tasks after proper normalization.

[3] To make the model meaningful, we assume $D_i \ge 2$ for each user $i \in \mathcal{K}$.

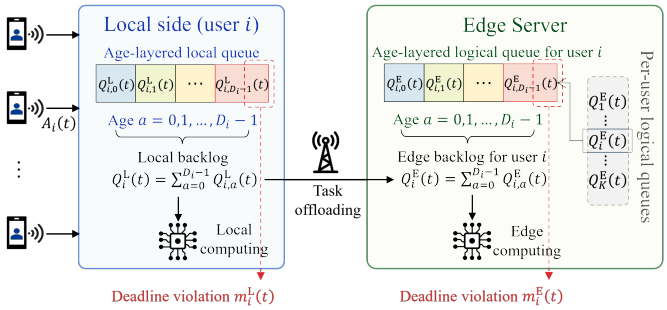


Fig. 1: Age-layered queueing architecture for deadline-aware MEC.

much smaller than the input data, the downlink feedback delay is neglected, which is standard in MEC modeling.

To explicitly model the deterministic end-to-end delay requirement, we adopt the age-layered queue representation with age index $a \in \{0, 1, \ldots, D_i - 1\}$ for each user $i \in \mathcal{K}$, where D_i is the delay bound in slots. An age-a workload has stayed in the system for a slots. Hence, any workload in layer $a = D_i - 1$ that is not completed within the current slot will violate the deterministic delay bound and thus become overdue. Such overdue workload is either removed from the system or immediately processed through an urgent computation service.

For each user i, let $Q^{\mathrm{L}}_{i,a}(t)$ and $Q^{\mathrm{E}}_{i,a}(t)$ denote the local and edge workloads of age a at the beginning of slot t, respectively. The corresponding total local and edge backlogs are

$$Q^{\mathrm{L}}_i(t) = \sum_{a=0}^{D_i-1} Q^{\mathrm{L}}_{i,a}(t), \qquad Q^{\mathrm{E}}_i(t) = \sum_{a=0}^{D_i-1} Q^{\mathrm{E}}_{i,a}(t). \tag{8}$$

Fig. 1 illustrates the proposed age-layered queueing architecture.

C. Age-priority Service Discipline

For age $a < D_i$, let $x_{i,a}(t)$ denote the amount of age-a workload locally processed by user i in slot t, and let $u_{i,a}(t)$ denote the amount of age-a workload offloaded in slot t. Similarly, let $y_{i,a}(t)$ denote the amount of age-a workload of user i processed at the edge in slot t. These *age-wise service variables* satisfy

$$x_{i,a}(t),\ u_{i,a}(t),\ y_{i,a}(t) \geq 0, \tag{9}$$

and

$$\sum_{a=0}^{D_i-1} x_{i,a}(t) \leq \mu^{\mathrm{L}}_i(t), \tag{10}$$

$$\sum_{a=0}^{D_i-1} u_{i,a}(t) \leq \mu^{\mathrm{T}}_i(t), \tag{11}$$

$$\sum_{a=0}^{D_i-1} y_{i,a}(t) \leq \mu^{\mathrm{E}}_i(t), \tag{12}$$

where $\mu^{\mathrm{L}}_i(t)$, $\mu^{\mathrm{T}}_i(t)$, and $\mu^{\mathrm{E}}_i(t)$ are the local computing, offload transmission, and edge computing capacities, respectively.

We impose an *age-priority service discipline* that always serves older workloads first. At the local side, in each slot, user i first allocates local computation to the largest-age layers and only then uses the residual transmission capacity to offload younger workloads. At the edge side, the server also processes larger-age workloads first. This rule is natural under hard deadline constraints and is without loss of modeling generality, and it is optimal for minimizing recourse under deterministic deadlines, similar to earliest-deadline-first (EDF).

The the age-wise service variables $x_{i,a}(t)$, $u_{i,a}(t)$, and $y_{i,a}(t)$ are uniquely induced by the age-priority service discipline as follows.

1) Local computing: Specifically, for each user i, local computing is performed first in descending age order. For $a = D_i - 1$ we have

$$x_{i,D_i-1}(t) = \min\left\{Q^{\mathrm{L}}_{i,D_i-1}(t), \mu^{\mathrm{L}}_i(t)\right\},\ \forall i, t, \tag{13}$$

and for $a = D_i - 2, D_i - 3, \ldots, 0$ we have

$$x_{i,a}(t) = \min\left\{Q^{\mathrm{L}}_{i,a}(t), \mu^{\mathrm{L}}_i(t) - \sum_{\alpha=a+1}^{D_i-1} x_{i,\alpha}(t)\right\},\ \forall i, t \tag{14}$$

2) Offloading: After local computing decisions are completed, the remaining local backlog becomes

$$\hat{Q}^{\mathrm{L}}_{i,a}(t) = Q^{\mathrm{L}}_{i,a}(t) - x_{i,a}(t),\ \forall a, i, t. \tag{15}$$

We prohibits offloading of age-$(D_i - 1)$ workload to prevent deadline violation during transmission. Therefore, we impose

$$u_{i,D_i-1}(t) = 0, \qquad \forall i, t. \tag{16}$$

For the remaining age layers, offloading is also carried out in descending age order over the residual eligible local backlog. Specifically, for $a = D_i - 2$ we have

$$u_{i,D_i-2}(t) = \min\left\{\hat{Q}^{\mathrm{L}}_{i,D_i-2}(t), \mu^{\mathrm{T}}_i(t)\right\},\ \forall i, t, \tag{17}$$

and for $a = D_i - 3, D_i - 4, \ldots, 0$ we have

$$u_{i,a}(t) = \min\left\{\hat{Q}^{\mathrm{L}}_{i,a}(t), \mu^{\mathrm{T}}_i(t) - \sum_{\alpha=a+1}^{D_i-2} u_{i,\alpha}(t)\right\},\ \forall i, t \tag{18}$$

3) Edge computing: At the edge side, the server also follows the oldest-first rule for each user's logical edge queue. Specifically, for $a = D_i - 1$ we have

$$y_{i,D_i-1}(t) = \min\left\{Q^{\mathrm{E}}_{i,D_i-1}(t), \mu^{\mathrm{E}}_i(t)\right\},\ \forall i, t, \tag{19}$$

and for $a = D_i - 2, D_i - 3, \ldots, 0$ we have

$$y_{i,a}(t) = \min\left\{Q^{\mathrm{E}}_{i,a}(t), \mu^{\mathrm{E}}_i(t) - \sum_{\alpha=a+1}^{D_i-1} y_{i,\alpha}(t)\right\},\ \forall i, t \tag{20}$$

Under the above construction, the age-wise service variables are uniquely determined by the aggregate capacities and queue states.

D. Deadline-Violation Modeling

For user i, define the local and edge deadline-violation amounts in slot t as

$$m_i^{\mathrm{L}}(t) = Q_{i,D_i-1}^{\mathrm{L}}(t) - x_{i,D_i-1}(t), \tag{21}$$

$$m_i^{\mathrm{E}}(t) = Q_{i,D_i-1}^{\mathrm{E}}(t) - y_{i,D_i-1}(t). \tag{22}$$

Note that under the age-priority service discipline, we have

$$m_i^{\mathrm{L}}(t) = \left[Q_{i,D_i-1}^{\mathrm{L}}(t) - \mu_i^{\mathrm{L}}(t)\right]^+, \tag{23}$$

$$m_i^{\mathrm{E}}(t) = \left[Q_{i,D_i-1}^{\mathrm{E}}(t) - \mu_i^{\mathrm{E}}(t)\right]^+, \tag{24}$$

where $[\cdot]^+ = \max\{\cdot, 0\}$. The quantity $m_i(t)$ measures the number of workload bits that miss the deterministic delay bound in slot t. Such overdue workload is removed from the system or forcibly completed through an urgent computation service. Each bit of deadline violation on the user device incurs a violation penalty of $\phi_i^{\mathrm{L}} > 0$. Similarly, each bit of deadline violation on the edge server incurs a violation penalty of $\phi_i^{\mathrm{E}} > 0$. The total deadline violation cost is $\phi_i^{\mathrm{L}} m_i^{\mathrm{L}}(t) + \phi_i^{\mathrm{E}} m_i^{\mathrm{E}}(t)$.

The quantity $m_i(t)$ denotes the number of workload bits that violate the deterministic delay bound in slot t. Such overdue workload is either removed from the system or processed through an urgent computation service. Each bit of deadline violation at the user device incurs a penalty $\phi_i^{\mathrm{L}} > 0$, while each bit of deadline violation at the edge server incurs a penalty $\phi_i^{\mathrm{E}} > 0$. Accordingly, the total deadline violation cost is given by

$$r(t) = \phi_i^{\mathrm{L}} m_i^{\mathrm{L}}(t) + \phi_i^{\mathrm{E}} m_i^{\mathrm{E}}(t) \tag{25}$$

E. Queue Evolution

1) Local queue dynamics: New arrivals enter the age-0 local queue at the beginning of each slot:

$$Q_{i,0}^{\mathrm{L}}(t) = A_i(t), \qquad \forall i, t. \tag{26}$$

During slot t, a portion of age-a workload is either processed locally, denoted by $x_{i,a}(t)$, or offloaded to the edge server, denoted by $u_{i,a}(t)$. The remaining workload ages by one slot. Therefore, for $a = 0, 1, \ldots, D_i - 2$, the queue evolution is given by

$$Q_{i,a+1}^{\mathrm{L}}(t+1) = Q_{i,a}^{\mathrm{L}}(t) - x_{i,a}(t) - u_{i,a}(t). \tag{27}$$

By summing (26) and (27) over all age layers, and using (10), (11), and (21), the total local backlog evolves as

$$\begin{aligned} Q_i^{\mathrm{L}}(t+1) =& Q_i^{\mathrm{L}}(t) - m_i^{\mathrm{L}}(t) + A_i(t+1) \\ &- \sum_{a=0}^{D_i-1} x_{i,a}(t) - \sum_{a=0}^{D_i-1} u_{i,a}(t) \\ \le& \left[Q_i^{\mathrm{L}}(t) - m_i^{\mathrm{L}}(t) - \mu_i^{\mathrm{L}}(t) - \mu_i^{\mathrm{T}}(t)\right]^+ \\ &+ A_i(t+1). \end{aligned} \tag{28}$$

2) Edge queue dynamics: Since no workload is directly generated at the edge, the age-0 edge layer is identically zero, i.e.,

$$Q_{i,0}^{\mathrm{E}}(t) = 0, \quad \forall i, t. \tag{29}$$

The offloaded workload experiences one-slot transmission delay. Specifically, the amount $u_{i,a}(t)$ offloaded from the local age-a queue in slot t arrives at the edge server at the beginning of slot $t+1$ with age $a+1$. Meanwhile, edge computing processes $y_{i,a}(t)$ amount of workload from the age-a queue. Therefore, for $a = 0, 1, \ldots, D_i - 2$, the edge queue evolves as

$$Q_{i,a+1}^{\mathrm{E}}(t+1) = Q_{i,a}^{\mathrm{E}}(t) - y_{i,a}(t) + u_{i,a}(t). \tag{30}$$

By summing (30) and (27) over all age layers, and using (11), (12), and (22), the total edge backlog evolves as Similarly, summing (30) over all age layers yields

$$\begin{aligned} Q_i^{\mathrm{E}}(t+1) &= Q_i^{\mathrm{E}}(t) - m_i^{\mathrm{E}}(t) - \sum_{a=0}^{D_i-1} y_{i,a}(t) + \sum_{a=0}^{D_i-1} u_{i,a}(t) \\ &\le \left[Q_i^{E}(t) - m_i^{E}(t) - \mu_i^{\mathrm{E}}(t)\right]^+ + \mu_i^{\mathrm{T}}(t). \end{aligned} \tag{31}$$

F. Feasible Control Policy

In each slot t, the system controller observes the network state

$$\boldsymbol{\Theta}(t) \triangleq \left\{Q_{i,a}^{\mathrm{L}}(t), Q_{i,a}^{\mathrm{E}}(t), A_i(t), H_i(t),\ \forall i, a\right\}, \tag{32}$$

and makes an online decision on local CPU frequencies, transmit powers, edge CPU allocation, and age-dependent service/offloading variables, i.e.,

$$\chi(t) \triangleq \left\{f_i^{\mathrm{L}}(t), p_i(t), f_i^{\mathrm{E}}(t),\ \forall i, a\right\}. \tag{33}$$

The age-wise service variables $\{x_{i,a}(t), u_{i,a}(t), y_{i,a}(t),\ \forall i, a\}$ are implicitly determined by the age-priority service discipline.

The objective of online optimization is to dynamically adapt $\chi(t)$ to stochastic arrivals and time-varying channels while guaranteeing the deterministic delay requirement. Depending on the design target, the system may minimize a long-term cost such as weighted communication-computation energy consumption, edge resource usage, or a network-wide utility loss, while preserving queue feasibility and deadline compliance.

IV. Problem Formulation

Under the system model developed in Sec. III, the objective is to design an online control policy that dynamically determines $\chi(t)$ in each slot to minimize the long-term system cost while guaranteeing deterministic delay.

A. Per-Slot Cost Function

In each slot t, the system incurs cost from three regular operations, namely local computing, wireless offloading, and

edge computing, as well as the deadline violation cost. Accordingly, the per-slot system cost is defined as

$$C(t) = \sum_{i\in\mathcal{K}} \Big(C_i^{\mathrm{L}}(t) + C_i^{\mathrm{T}}(t) + C_i^{\mathrm{E}}(t) \Big) + \sum_{i\in\mathcal{K}} \Big(\phi_i^{\mathrm{L}} m_i^{\mathrm{L}}(t) + \phi_i^{\mathrm{E}} m_i^{\mathrm{E}}(t) \Big), \tag{34}$$

where $C_i^{\mathrm{L}}(t)$, $C_i^{\mathrm{T}}(t)$, $C_i^{\mathrm{E}}(t)$ are local computing cost, transmission cost and edge computing cost, respectively.

1) Local computing cost: For user i, the local CPU frequency in slot t is denoted by $f_i^{\mathrm{L}}(t)$. Following the standard DVFS model, the local computing energy consumption is proportional to the square of the CPU frequency per cycle, yielding the per-slot local computing cost

$$C_i^{\mathrm{L}}(t) = \delta\kappa\big(f_i^{\mathrm{L}}(t)\big)^3, \tag{35}$$

where $\kappa > 0$ is the effective switched-capacitance coefficient.

2) Transmission cost: Let $p_i(t)$ denote the uplink transmit power of user i in slot t. The transmission cost is defined as the uplink transmission energy consumed during one slot:

$$C_i^{\mathrm{T}}(t) = \delta p_i(t). \tag{36}$$

This expression explicitly captures the increasing energy cost required to support larger offloading volumes under a given channel state.

3) Edge computing cost: In slot t, the edge computing energy consumption for processing workload from user i is

$$C_i^{\mathrm{E}}(t) = \delta\kappa\big(f_i^{\mathrm{E}}(t)\big)^3, \tag{37}$$

where $\kappa > 0$ is the effective switched-capacitance coefficient at the edge.

The long-term optimization aims to balance local energy consumption, transmission energy consumption, and deadline violation cost under stochastic arrivals and time-varying wireless channels. Therefore, the proposed model not only guarantees deterministic delay but also preserves a meaningful tradeoff between deadline reliability and operational cost.

B. Long-Term Optimization Problem

We define the long-term time-average expectation of system cost $\bar{C}$ and total queue backlog $\bar{Q}$ as follows:

$$\bar{C} = \limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[C(t)\right], \tag{38}$$

$$\bar{Q} = \limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i\in\mathcal{K}} \left(\mathbb{E}\left[Q_i^{\mathrm{L}}(t)\right] + \mathbb{E}\left[Q_i^{\mathrm{E}}(t)\right] \right). \tag{39}$$

Based on the per-slot cost defined in (34), we aim to design an online control policy that minimizes the long-term time-average system cost.

Specifically, let $\{\chi(t)\}_{t=0}^{\infty}$ denote a sequence of feasible control actions adapted to the observed network state $\boldsymbol{\Theta}(t)$. The long-term optimization problem is formulated as

$$\textbf{(P1)} \quad \min_{\{\chi(t)\}} \quad \bar{C}$$
$$\text{s.t.} \quad \bar{Q} < \infty,$$
$$(3),\ (4),\ (5),$$
$$\text{Age-priority service discipline (Sec. III-C).}$$

Here, the expectation is taken with respect to the randomness of arrivals and wireless channel states.

Due to the stochastic nature and the coupled dynamics, problem **(P1)** is difficult to solve directly. In the next section, we develop a Lyapunov optimization framework to derive an efficient online algorithm that makes per-slot decisions while provably minimizing the long-term cost.

V. Algorithm Design

A. Lyapunov Framework

To solve problem **(P1)**, we define the quadratic Lyapunov function as

$$L(t) = \frac{1}{2} \sum_{i\in\mathcal{K}} \Big[\big(Q_i^{\mathrm{L}}(t)\big)^2 + \big(Q_i^{\mathrm{E}}(t)\big)^2 \Big]. \tag{40}$$

The one-slot conditional Lyapunov drift is

$$\Delta(t) = \mathbb{E}\big[L(t+1) - L(t) \mid \boldsymbol{Q}(t)\big], \tag{41}$$

where $\boldsymbol{Q}(t) = [Q_1^{\mathrm{L}}(t), \ldots, Q_K^{\mathrm{L}}(t), Q_1^{\mathrm{E}}(t), \ldots, Q_K^{\mathrm{E}}(t)]$.

Using standard Lyapunov drift analysis, there exists a constant $B > 0$ such that

$$\Delta(t) \le B + \mathbb{E}\big[\Xi(t) \mid \boldsymbol{Q}(t)\big], \tag{42}$$

where

$$\Xi(t) = \sum_{i\in\mathcal{K}} Q_i^{\mathrm{L}}(t)\big(A_i(t) - \mu_i^{\mathrm{L}}(t) - \mu_i^{\mathrm{T}}(t) - m_i^{\mathrm{L}}(t)\big) + \sum_{i\in\mathcal{K}} Q_i^{\mathrm{E}}(t)\big(\mu_i^{\mathrm{T}}(t) - \mu_i^{\mathrm{E}}(t) - m_i^{\mathrm{E}}(t)\big). \tag{43}$$

B. Drift-Plus-Penalty Minimization

Define the drift-plus-penalty as

$$\Delta_V(t) = \Delta(t) + V\,\mathbb{E}[C(t) \mid \boldsymbol{Q}(t)], \tag{44}$$

where $V > 0$ is the Lyapunov parameter. By substituting (42) into (44), we have

$$\Delta_V(t) \le B + \mathbb{E}\big[\Psi(t) \mid \boldsymbol{Q}(t)\big], \tag{45}$$

where

$$\Psi(t) = \Xi(t) + VC(t). \tag{46}$$

Minimizing the upper bound yields the per-slot problem

$$\textbf{(P2)} \quad \min_{\boldsymbol{\chi}(t)} \quad \Psi(t). \tag{47}$$

C. Problem Decomposition

Problem **(P2)** reveals an important separable structure. Specifically, local computing and transmission decisions are decoupled across users, while edge CPU allocation is coupled due to the shared resource constraint.

1) Local CPU optimization: For each user i, the local CPU allocation is obtained by solving

$$\begin{aligned} \min_{f_i^{\mathrm{L}}(t)} \quad & VC_i^{\mathrm{L}}(t) + V\phi_i^{\mathrm{L}} m_i^{\mathrm{L}}(t) - Q_i^{\mathrm{L}}(t)\left(\mu_i^{\mathrm{L}}(t) + m_i^{\mathrm{L}}(t)\right) \\ \text{s.t.} \quad & (1)(3)(23)(35). \end{aligned}$$

Define

$$f_{i,\mathrm{th}}^{\mathrm{L}} = c_i Q_{i,D_i-1}^{\mathrm{L}}(t)/\delta. \tag{48}$$

By considering the two regions induced by the $\max\{\cdot\}$ operator, the optimal solution can be obtained as

$$\tilde{f}_i^{\mathrm{L}}(t) = \arg \min_{f \in \{f_{i,1}^{\mathrm{L}},\, f_{i,2}^{\mathrm{L}}\}} J_i^{\mathrm{L}}(f), \tag{49}$$

where

$$f_{i,1}^{\mathrm{L}} = \left[\sqrt{\phi_i^{\mathrm{L}}/(3\kappa c_i)}\right]_0^{\min\{f_{i,\max}^{\mathrm{L}},\, f_{i,\mathrm{th}}^{\mathrm{L}}\}}, \tag{50}$$

$$f_{i,2}^{\mathrm{L}} = \left[\sqrt{Q_i^{\mathrm{L}}(t)/(3V\kappa c_i)}\right]_{f_{i,\mathrm{th}}^{\mathrm{L}}}^{f_{i,\max}^{\mathrm{L}}}, \tag{51}$$

and

$$\begin{aligned} J_i^{\mathrm{L}}(f) &= V\delta\kappa f^3 - \delta Q_i^{\mathrm{L}}(t) f/c_i \\ &\quad + \left(V\phi_i^{\mathrm{L}} - Q_i^{\mathrm{L}}(t)\right) \max\left\{0, Q_{i,D_i-1}^{\mathrm{L}}(t) - \delta f/c_i\right\}. \end{aligned} \tag{52}$$

Here, $[x]_a^b \triangleq \min\{\max\{x,a\},b\}$, and only feasible candidates are retained.

2) Transmission power optimization: For each user i, the transmission power is obtained by solving

$$\begin{aligned} \min_{p_i(t)} \quad & VC_i^{\mathrm{T}}(t) + \left(Q_i^{\mathrm{E}}(t) - Q_i^{\mathrm{L}}(t)\right)\mu_i^{\mathrm{T}}(t) \\ \text{s.t.} \quad & (5)(6)(36). \end{aligned}$$

The optimal solution is obtained in closed form as

$$\tilde{p}_i(t) = \left[\left(Q_i^{\mathrm{L}}(t) - Q_i^{\mathrm{E}}(t)\right)B_i/(V\ln 2) - N_0 B_i/H_i(t)\right]_0^{p_{i,\max}}, \tag{53}$$

where $[x]_a^b \triangleq \min\{\max\{x,a\},b\}$.

3) Edge CPU allocation: The edge computing resource allocation is obtained by solving

$$\begin{aligned} \min_{\{f_i^{\mathrm{E}}(t)\}} \sum_{i\in\mathcal{K}} & VC_i^{\mathrm{E}}(t) + V\phi_i^{\mathrm{E}} m_i^{\mathrm{E}}(t) - Q_i^{\mathrm{E}}(t)\left(\mu_i^{\mathrm{E}}(t) + m_i^{\mathrm{E}}(t)\right) \\ \text{s.t.} \quad & (2)(4)(23)(37). \end{aligned}$$

To solve the edge CPU allocation problem, we first introduce the dual variable $\lambda \geq 0$ associated with the coupling constraint $\sum_{i\in\mathcal{K}} f_i^{\mathrm{E}}(t) \leq f_{\max}^{\mathrm{E}}$. The resulting Lagrangian can be written as

$$\mathcal{L}\left(\{f_i^{\mathrm{E}}(t)\},\lambda\right) = \sum_{i\in\mathcal{K}} \left(J_i^{\mathrm{E}}\left(f_i^{\mathrm{E}}(t)\right) + \lambda f_i^{\mathrm{E}}(t)\right) - \lambda f_{\max}^{\mathrm{E}}, \tag{54}$$

Algorithm 1 Bisection Search for Optimal Dual Variable λ^*

Initialize $\lambda_{\min} = 0$, $\lambda_{\max} = \max_i \frac{\delta}{c_i}\max\{V\phi_i^{\mathrm{E}}, Q_i^{\mathrm{E}}(t)\}$, and tolerance $\epsilon > 0$.
repeat
  Set $\lambda = (\lambda_{\min} + \lambda_{\max})/2$.
  for each $i \in \mathcal{K}$ **do**
    Compute $\tilde{f}_i^{\mathrm{E}}(t;\lambda)$ according to (57).
  end for
  if $\sum_{i\in\mathcal{K}} \tilde{f}_i^{\mathrm{E}}(t;\lambda) > f_{\max}^{\mathrm{E}}$ **then**
    Update $\lambda_{\min} \leftarrow \lambda$.
  else
    Update $\lambda_{\max} \leftarrow \lambda$.
  end if
until $|\lambda_{\max} - \lambda_{\min}| < \epsilon$
Set $\lambda^* = \lambda$.
Output $\tilde{f}_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t;\lambda^*)$, $\forall i \in \mathcal{K}$.

where

$$\begin{aligned} J_i^{\mathrm{E}}(f) &= V\delta\kappa f^3 - \delta Q_i^{\mathrm{E}}(t) f/c_i \\ &\quad + \left(V\phi_i^{\mathrm{E}} - Q_i^{\mathrm{E}}(t)\right)\left[Q_{i,D_i-1}^{\mathrm{E}}(t) - \delta f/c_i\right]^+. \end{aligned} \tag{55}$$

For a given λ, the problem is decomposed into a set of independent one-dimensional subproblems. Define

$$f_{i,\mathrm{th}}^{\mathrm{E}} = c_i Q_{i,D_i-1}^{\mathrm{E}}(t)/\delta. \tag{56}$$

By considering the two regions induced by the $\max\{\cdot\}$ operator, the optimal solution of the ith subproblem is given by

$$\tilde{f}_i^{\mathrm{E}}(t;\lambda) = \arg \min_{f \in \{f_{i,1}^{\mathrm{E}}(\lambda),\, f_{i,2}^{\mathrm{E}}(\lambda)\}} \left(J_i^{\mathrm{E}}(f) + \lambda f\right), \tag{57}$$

where

$$f_{i,1}^{\mathrm{E}}(\lambda) = \left[\sqrt{[V\delta\phi_i^{\mathrm{E}}/c_i - \lambda]^+/(3V\delta\kappa)}\right]_0^{f_{i,\mathrm{th}}^{\mathrm{E}}}, \tag{58}$$

$$f_{i,2}^{\mathrm{E}}(\lambda) = \left[\sqrt{[\delta Q_i^{\mathrm{E}}(t)/c_i - \lambda]^+/(3V\delta\kappa)}\right]_{f_{i,\mathrm{th}}^{\mathrm{E}}}^{+\infty}, \tag{59}$$

Accordingly, the optimal solution is obtained as

$$\tilde{f}_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t;\lambda^*), \quad \forall i \in \mathcal{K}, \tag{60}$$

where the optimal dual variable λ^* is chosen such that

$$\sum_{i\in\mathcal{K}} \tilde{f}_i^{\mathrm{E}}(t;\lambda^*) \leq f_{\max}^{\mathrm{E}}, \tag{61}$$

with equality holding when $\lambda^* > 0$.

Since $\sum_{i\in\mathcal{K}} \tilde{f}_i^{\mathrm{E}}(t;\lambda)$ is monotonically non-increasing with respect to λ, the optimal dual variable λ^* can be efficiently obtained via a one-dimensional bisection search, as summarized in Algorithm 1.

The complete online policy is summarized in Algorithm 2. Its per-slot complexity is $O(K)$ for local CPU and power updates, and $O(K\log(1/\epsilon))$ for edge CPU allocation via bisection.

Algorithm 2 Age-layered Deterministic-Delay Optimization (ADDO) Algorithm

for each slot $t = 0, 1, 2, \ldots$ **do**
  Observe system state $\mathbf{\Theta}(t)$.
  Compute λ^* via Algorithm 1.
  for each user $i \in \mathcal{K}$ **do**
    Compute $\tilde{f}_i^{\mathrm{L}}(t)$ according to (49), set $f_i^{\mathrm{L}}(t) = \tilde{f}_i^{\mathrm{L}}(t)$.
    Compute $\tilde{p}_i(t)$ according to (53), set $p_i(t) = \tilde{p}_i(t)$.
    Compute $\tilde{f}_i^{\mathrm{E}}(t; \lambda^*)$ according to (57), set $f_i^{\mathrm{E}}(t) = \tilde{f}_i^{\mathrm{E}}(t; \lambda^*)$.
    Serve workload following the age-priority service discipline.
  end for
  Update queues according to (26)–(31).
end for

D. Performance Guarantee

1) Cost and Backlog Upper Bound:

Theorem 1: Assume arrival $\{A_i(t),\ \forall i, t\}$ lies in the interior of the capacity region of the system. Let C^* denote the minimum time-average expected cost of problem **(P1)** over all feasible policies. Then, under ADDO with control parameter $V > 0$, the following holds:

$$\bar{C} \leq C^* + B/V, \tag{62}$$

$$\bar{Q} \leq \min\left\{(B + VC_{\max})/\varepsilon, \sum_{i\in\mathcal{K}} D_i A_{i,\max}\right\}, \tag{63}$$

where $B, \epsilon > 0$, and

$$C_{\max} \triangleq \delta\kappa(f_{\max}^{\mathrm{E}})^3 + \delta\sum_{i\in\mathcal{K}}[\kappa(f_{i,\max}^{\mathrm{L}})^3 + p_{i,\max}] + \sum_{i\in\mathcal{K}}[(\phi_i^{\mathrm{L}} + \phi_i^{\mathrm{E}})A_{i,\max}] \tag{64}$$

is a finite per-slot cost upper bound.

Proof 1: ADDO minimizes the right-hand side of the following inequality over all feasible control actions in each slot:

$$\Delta_V(t) \leq B + \mathbb{E}\left[VC(t) + \Xi(t)|\mathbf{Q}(t)\right]. \tag{65}$$

Therefore, it performs no worse than any feasible stationary randomized policy. Since the arrival lies in the interior of the capacity region of the system, there exists an optimal scheme which makes i.i.d. decision $\chi^*(t)$ over time independent of the queue backlog sizes such that

$$\mathbb{E}[A_i(t)] \leq \mathbb{E}[\mu_i^{\mathrm{L}}(t) + \mu_i^{\mathrm{T}}(t) + m_i^{\mathrm{L}}(t) \mid \chi^*(t)], \tag{66}$$

$$\mathbb{E}[\mu_i^{\mathrm{T}}(t) \mid \chi^*(t)] \leq \mathbb{E}[\mu_i^{\mathrm{E}}(t) + m_i^{\mathrm{E}}(t) \mid \chi^*(t)]. \tag{67}$$

By (43), we have

$$\mathbb{E}\left[\Xi(t) \mid \boldsymbol{Q}(t), \chi^*(t)\right] \leq 0. \tag{68}$$

Since ADDO minimizes the right-hand side of (65), we have

$$\begin{aligned}\Delta_V(t) &\leq B + \mathbb{E}\left[VC(t) + \Xi(t) \mid \boldsymbol{Q}(t), \chi^*(t)\right]\\ &= B + \mathbb{E}\left[VC(t) \mid \boldsymbol{Q}(t), \chi^*(t)\right]\\ &= B + VC^*.\end{aligned} \tag{69}$$

Taking full expectation and summing over $t = 0, 1, \ldots, T-1$, we obtain

$$\mathbb{E}[L(T)] - \mathbb{E}[L(0)] + V\sum_{t=0}^{T-1}\mathbb{E}[C(t)] \leq TB + TVC^*. \tag{70}$$

Rearranging terms gives

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}[C(t)] \leq C^* + \frac{B}{V} + \frac{\mathbb{E}[L(0)] - \mathbb{E}[L(T)]}{VT}. \tag{71}$$

Since $\mathbb{E}[L(T)] \geq 0$ and $\mathbb{E}[L(0)] < \infty$, taking $T \to \infty$ yields (62).

For each user i, the total workload in age layer a satisfies

$$Q_{i,a}^L(t) + Q_{i,a}^E(t) \leq A_{i,\max},\ a = 0, 1, \ldots, D_i - 1. \tag{72}$$

Consequently, the total backlog of user i is deterministically bounded as

$$Q_i^L(t) + Q_i^E(t) \leq D_i A_{i,\max},\ \forall t, \tag{73}$$

Taking long-term time-averaged expectation of the left-hand side of above inequality, we have

$$\bar{Q} \leq \sum_{i\in\mathcal{K}} D_i A_{i,\max}. \tag{74}$$

Since the arrival lies in the interior of the capacity region of the system, there must exist some $\varepsilon > 0$ such that exists a feasible scheme which makes i.i.d. decision $\chi^\varepsilon(t)$ over time independent of the queue backlog sizes such that

$$\mathbb{E}[A_i(t)] \leq \mathbb{E}[\mu_i^{\mathrm{L}}(t) + \mu_i^{\mathrm{T}}(t) + m_i^{\mathrm{L}}(t) \mid \chi^\varepsilon(t)] - \varepsilon, \tag{75}$$

$$\mathbb{E}[\mu_i^{\mathrm{T}}(t) \mid \chi^\varepsilon(t)] \leq \mathbb{E}[\mu_i^{\mathrm{E}}(t) + m_i^{\mathrm{E}}(t) \mid \chi^\varepsilon(t)] - \varepsilon. \tag{76}$$

By (43), we have

$$\mathbb{E}\left[\Xi(t) \mid \boldsymbol{Q}(t), \chi^\varepsilon(t)\right] \leq \varepsilon\sum_{i\in\mathcal{K}}\left(Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right). \tag{77}$$

Since ADDO minimizes the right-hand side of (65), we have

$$\begin{aligned}\Delta_V(t) \leq& B + \mathbb{E}\left[VC(t) + \Xi(t) \mid \Theta(t)\right]\\ \leq& B + \mathbb{E}\left[VC(t) + \Xi(t) \mid \Theta(t), \chi^\varepsilon(t)\right]\\ \leq& B + \mathbb{E}\left[VC(t) \mid \boldsymbol{Q}(t), \chi^\varepsilon(t)\right]\\ &- \varepsilon\sum_{i\in\mathcal{K}}\left(Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right)\\ \leq& B + VC_{\max} - \varepsilon\sum_{i\in\mathcal{K}}\left(Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right).\end{aligned} \tag{78}$$

Taking full expectation, we obtain

$$\begin{aligned}&\mathbb{E}\left[L(t+1)\right] - \mathbb{E}\left[L(t)\right] + V\mathbb{E}\left[C(t)\right]\\ &\quad\leq B + VC_{\max} + \varepsilon\sum_{i\in\mathcal{K}}\mathbb{E}\left[Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right].\end{aligned} \tag{79}$$

Rearranging terms gives

$$\begin{aligned}
&\varepsilon \sum_{i\in\mathcal{K}} \mathbb{E}\left[Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right] \\
&\le B + VC_{\max} - V\mathbb{E}\left[C(t)\right] + \mathbb{E}\left[L(t)\right] - \mathbb{E}\left[L(t+1)\right] \\
&\le B + VC_{\max} + \mathbb{E}\left[L(t)\right] - \mathbb{E}\left[L(t+1)\right]. \qquad (80)
\end{aligned}$$

Summing over $t = 0, 1, \ldots, T-1$, we obtain

$$\begin{aligned}
&\frac{1}{T}\sum_{t=0}^{T-1}\sum_{i\in\mathcal{K}} \mathbb{E}\left[Q_i^{\mathrm{L}}(t) + Q_i^{\mathrm{E}}(t)\right] \\
&\qquad \le \frac{B + VC_{\max}}{\varepsilon} + \frac{\mathbb{E}\left[L(0)\right] - \mathbb{E}\left[L(T)\right]}{\varepsilon T}. \qquad (81)
\end{aligned}$$

Since $\mathbb{E}[L(T)] \ge 0$ and $\mathbb{E}[L(0)] < \infty$, taking $T \to \infty$ yields

$$\bar{Q} \le \frac{B + VC_{\max}}{\varepsilon}. \qquad (82)$$

Combining with (74), (63) is proven.

2) Deadline Violations:

Theorem 2 (Average deadline-violation bound): Define the time-average expected deadline-violation amount as

$$\bar{M} \triangleq \limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_{i\in\mathcal{K}} \mathbb{E}\big[m_i^{\mathrm{L}}(t) + m_i^{\mathrm{E}}(t)\big]. \qquad (83)$$

Let

$$\phi_{\min} \triangleq \min_{i\in\mathcal{K}}\{\phi_i^{\mathrm{L}}, \phi_i^{\mathrm{E}}\} > 0. \qquad (84)$$

Then, under ADDO,

$$\bar{M} \le \frac{\bar{C}}{\phi_{\min}} \le \frac{C^*}{\phi_{\min}} + \frac{B}{V\phi_{\min}}. \qquad (85)$$

Proof 2: By the per-slot cost definition (34), we have

$$\begin{aligned}
C(t) &\ge \sum_{i\in\mathcal{K}} \left(\phi_i^{\mathrm{L}} m_i^{\mathrm{L}}(t) + \phi_i^{\mathrm{E}} m_i^{\mathrm{E}}(t)\right) \\
&\ge \phi_{\min} \sum_{i\in\mathcal{K}} \left(m_i^{\mathrm{L}}(t) + m_i^{\mathrm{E}}(t)\right). \qquad (86)
\end{aligned}$$

Taking expectation, time average, and $\limsup$ on both sides yields

$$\bar{M} \le \bar{C}/\phi_{\min}. \qquad (87)$$

Combining this with (62) proves (85).

VI. Numerical Results

A. Simulation Setup

We consider a stochastic MEC system consisting of a single edge server and 20 user devices. Time is slotted, and each slot has duration $\delta = 10$ ms. For each user i, the workload arrival process $\{A_i(t)\}$ is modeled as an independent and identically distributed (i.i.d.) random process, where $A_i(t) \sim \mathrm{Unif}(0, 3\times 10^4)$ bits. Each workload bit requires $c_i \sim \mathrm{Unif}(500, 1000)$ CPU cycles. All tasks are subject to a common deterministic delay bound $D_i = D$, and we set $D = 4$ slots throughout the simulations.

For the wireless uplink, we adopt the frequency-flat block fading model introduced in Sec. III. The reference channel power gain is set to $\beta_0 = 10^{-4}$ (i.e., -40 dB), and the path-loss exponent is $\alpha = 4$. Unless otherwise specified, all users are located at a fixed distance of 150 m from the edge node. The noise power spectral density is set to $N_0 = -174$ dBm/Hz. The uplink bandwidth of each user is $B_i = 1$ MHz, and the maximum transmit power is $p_{i,\max} = 100$ mW.

Each user is equipped with a local processor with maximum CPU frequency $f_{i,\max}^L = 1$ GHz. The edge server has a total computation capacity of $f_{\max}^E = 10$ GHz shared by all users. The effective switched-capacitance coefficient in the DVFS power model is set to $\kappa = 10^{-28}$.

To emphasize deterministic latency protection, the deadline-violation penalty coefficients are set to $\phi_i^L = \phi_i^E = 0.01$. We compare the proposed ADDO algorithm with four baseline policies: 1) a delay-aware energy-efficient (DAEE) scheme [20], which guarantees task deadlines by constructing virtual queues and using perturbed Lyapunov optimization techniques; 2) a local-only policy, where all workloads are processed locally; 3) an edge-only policy, where all workloads are offloaded to the edge server; and 4) a random policy, which allocates communication and computation resources uniformly over the feasible action space.

The system performance is evaluated using three metrics: 1) the average energy cost (J/slot); 2) the deadline-violation ratio, defined as the ratio of the average overdue workload to the total arrived workload; and 3) the average delay per bit of workload, measured in slots.

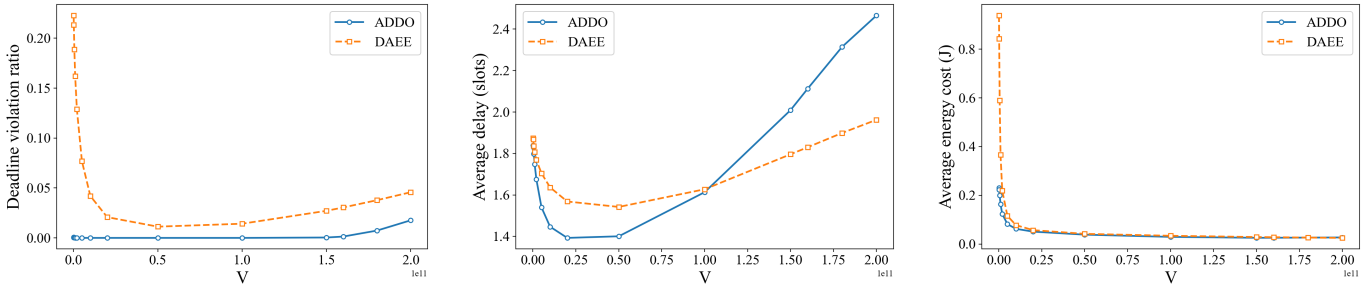
TABLE I: Performance Comparison of Different Policies

Policy	Energy Cost (J/slot)	Deadline Violation Ratio	Average Delay (slots/bit)
ADDO	0.0296	**0.0000**	**1.6135**
DAEE	0.0341	0.0142	1.6276
LocalOnly	**0.0200**	0.1544	3.0029
EdgeOnly	0.0258	0.5642	3.9196
Random	0.0282	0.2373	2.8050

B. Experimental Results

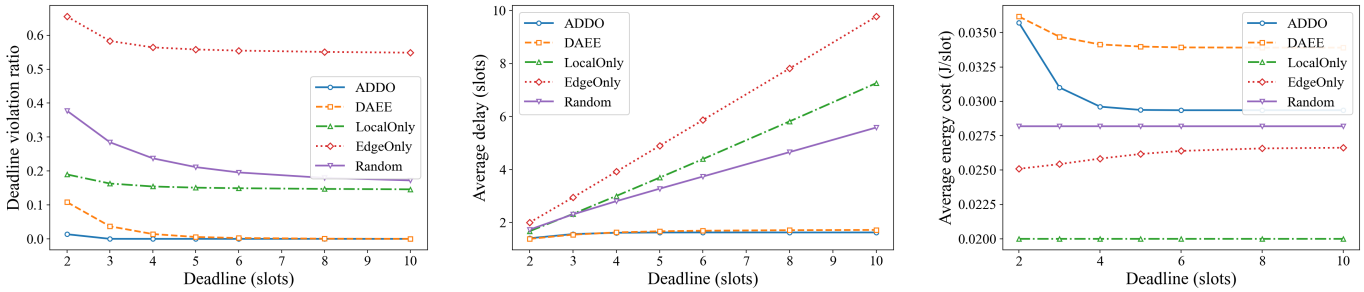
1) Comparison with baseline policies: Table I compares the proposed ADDO algorithm with four baseline policies, in which V is set to 10^{11} for ADDO and DAEE. It can be observed that ADDO achieves the lowest deadline violation ratio and average delay among all considered schemes. Compared with DAEE, ADDO reduces the average energy cost from 0.0341 to 0.0296 J/slot, corresponding to a reduction of approximately 13.20%, while simultaneously eliminating deadline violations and slightly lowering the average delay.

Although the LocalOnly, EdgeOnly and Random policies achieve lower energy costs than ADDO, their reliability degradation is substantial. Specifically, LocalOnly yields the lowest energy cost of 0.0200 J/slot, which is 32.43% lower than that of ADDO, but incurs a deadline-violation ratio of 0.1544 and an average delay of 3.0029 slots/bit. In comparison, ADDO completely eliminates deadline violations, while reducing the average delay by 46.27%. These results indicate that purely local execution or full offloading cannot effectively handle



(a) Deadline violation ratio. (b) Average delay. (c) Average energy cost.

Fig. 2: Performance versus the Lyapunov parameter V under ADDO and DAEE.



(a) Deadline violation ratio. (b) Average delay. (c) Average energy cost.

Fig. 3: Performance versus the deterministic deadline D under different policies.

stochastic arrivals and communication-computation coupling under hard deadline constraints.

2) Comparison of ADDO and DAEE under different V: Fig. 2 compares ADDO and DAEE as the Lyapunov parameter V varies from 10^8 to 2×10^{11}. As shown in Fig. 2a, ADDO achieves consistently better deadline reliability than DAEE. In particular, the deadline-violation ratio of ADDO remains almost zero for $V \leq 1.5 \times 10^{11}$, while DAEE still exhibits non-negligible violations. Even at $V = 2 \times 10^{11}$, ADDO reduces the violation ratio from 0.0457 to 0.0176, corresponding to a reduction of 61.49%.

Fig. 2b further shows that ADDO achieves lower delay than DAEE in the moderate-V regime. For example, at $V = 2 \times 10^{10}$, ADDO reduces the average delay from 1.5685 to 1.3930 slots, i.e., by 11.20%. However, when V becomes excessively large, the delay of ADDO increases more rapidly. Therefore, a moderate choice of V is preferred to balance delay performance and deadline reliability.

The corresponding energy cost is shown in Fig. 2c. For both algorithms, the average energy cost decreases with V and gradually saturates. In the small-V regime, ADDO is significantly more energy-efficient. For example, at $V = 10^8$, ADDO reduces the average energy cost from 0.9378 to 0.2309 J/slot, corresponding to a reduction of 75.39%. Overall, ADDO achieves a more favorable reliability–delay–energy tradeoff than DAEE.

3) Performance versus the deterministic deadline D: Fig. 3 shows the performance of different policies as the deadline D varies from 2 to 10 slots, in which V is set to 10^{11} for ADDO and DAEE. As shown in Fig. 3a, ADDO achieves the best deadline reliability over the entire range of D. In particular, the violation ratio of ADDO is about 0.0136 at $D = 2$ and drops to zero for $D \geq 3$, while DAEE still incurs a violation ratio of about 0.1082 at $D = 2$ and drops to 0.0001 until $D = 8$. In contrast, LocalOnly, EdgeOnly, and Random exhibit persistently high violation ratios even when the deadline is relaxed.

Fig. 3b shows that ADDO also maintains low delay as D increases. For example, at $D = 10$, the average delay of ADDO is about 1.6256 slots, compared with 1.7201 for DAEE, 7.2597 for LocalOnly, 9.7731 for EdgeOnly, and 5.5849 for Random. This indicates that ADDO effectively prevents excessive queue buildup while satisfying deadline constraints.

The corresponding energy cost is shown in Fig. 3c. The energy cost of ADDO decreases from about 0.0357 J/slot at $D = 2$ to about 0.0294 J/slot at $D \geq 5$, corresponding to a reduction of approximately 17.65%. Over the entire range of D, ADDO also consumes less energy than DAEE; for example, at $D = 10$, ADDO reduces the average energy cost from 0.0340 to 0.0294 J/slot, i.e., by about 13.53%. Overall, these results show that ADDO achieves a more favorable delay–energy–reliability tradeoff than the benchmark policies under different deterministic deadlines.

VII. Conclusion

This paper investigated online resource allocation for MEC networks under hard end-to-end delay constraints. To explicitly characterize workload urgency, we developed an age-layered queueing model for both local and edge queues, together with an age-priority service discipline for deadline-aware processing. Based on this model, we formulated a long-term stochastic optimization problem that jointly considers local computing energy, transmission energy, edge computing energy, and deadline-violation penalties. We further proposed the ADDO algorithm based on Lyapunov drift-plus-penalty optimization, which makes online decisions on local computing, task offloading, and edge CPU allocation without requiring prior knowledge of traffic-arrival or channel-state statistics. Theoretical analysis established an $O(1/V)$ cost-optimality gap. Numerical results demonstrated that ADDO achieves lower deadline-violation ratios and a more favorable delay–energy–reliability tradeoff than benchmark policies, and can eliminate deadline violations under proper parameter settings. Future work will extend the proposed framework to multi-edge-server scenarios with user mobility, heterogeneous service deadlines, and cooperative edge resource scheduling.

References

[1] C.-X. Wang, X. You, X. Gao, X. Zhu, Z. Li, C. Zhang, H. Wang, Y. Huang, Y. Chen, H. Haas *et al.*, "On the road to 6g: Visions, requirements, key technologies, and testbeds," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 905–974, 2023.

[2] M. Ergen, B. Saoud, I. Shayea, A. A. El-Saleh, O. Ergen, F. Inan, and M. F. Tuysuz, "Edge computing in future wireless networks: A comprehensive evaluation and vision for 6g and beyond," *ICT Express*, vol. 10, no. 5, pp. 1151–1173, 2024.

[3] J. Sasiain, D. Franco, A. Atutxa, J. Astorga, and E. Jacob, "Toward the integration and convergence between 5g and tsn technologies and architectures for industrial communications: A survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 1, pp. 259–321, 2025.

[4] J. Ren, G. Yu, Y. Cai, and Y. He, "Latency optimization for resource allocation in mobile-edge computation offloading," *IEEE Transactions on Wireless Communications*, vol. 17, no. 8, pp. 5506–5519, 2018.

[5] Q. Wang, X. Chen, and Q. Qi, "Design of delay-optimal robust edge computing in 6g wireless networks," in *2022 14th International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, 2022, pp. 226–231.

[6] Z. Nan, Y. Han, S. Zhou, and Z. Niu, "Robust dnn partitioning and resource allocation under uncertain inference time," *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3680–3696, 2026.

[7] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.

[8] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3590–3605, 2016.

[9] H. Chahed and A. Kassler, "Tsn network scheduling—challenges and approaches," *Network*, vol. 3, no. 4, pp. 585–624, 2023.

[10] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. ElBakoury, "Ultra-low latency (ull) networks: The ieee tsn and ietf detnet standards and related 5g ull research," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 88–145, 2018.

[11] B. S. Khan, S. Jangsher, A. Ahmed, and A. Al-Dweik, "Urllc and embb in 5g industrial iot: A survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1134–1163, 2022.

[12] C. Xu, Y. Liu, C. Wang, and A. Nallanathan, "Analysis of grant-free access for urllc service: A stochastic network calculus perspective," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 9, pp. 14 949–14 954, 2025.

[13] C.-F. Liu, M. Bennis, M. Debbah, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Transactions on Communications*, vol. 67, no. 6, pp. 4132–4150, 2019.

[14] S. Yue, J. Ren, N. Qiao, Y. Zhang, H. Jiang, Y. Zhang, and Y. Yang, "Todg: Distributed task offloading with delay guarantees for edge computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 7, pp. 1650–1665, 2021.

[15] J. Cao, W. Feng, N. Ge, and J. Lu, "Delay characterization of mobile-edge computing for 6g time-sensitive services," *IEEE Internet of Things Journal*, vol. 8, no. 5, pp. 3758–3773, 2020.

[16] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated mec systems: A distributed online approach," *IEEE Transactions on Mobile Computing*, vol. 23, no. 8, pp. 8129–8142, 2023.

[17] S. Xia, Z. Yao, Y. Li, Z. Xing, and S. Mao, "Distributed computing and networking coordination for task offloading under uncertainties," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5280–5294, 2024.

[18] L. Tang, M. Xu, C. Xu, and K. Ye, "Collaborative resource management and workloads scheduling in cloud-assisted mobile edge computing across timescales," in *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 1415–1425.

[19] S. Yao, M. Wang, J. Ren, T. Xia, W. Wang, K. Xu, M. Xu, and H. Zhang, "Multi-agent reinforcement learning for task offloading in crowd-edge computing," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9289–9302, 2025.

[20] H. Wu, J. Chen, T. N. Nguyen, and H. Tang, "Lyapunov-guided delay-aware energy efficient offloading in iiot-mec systems," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 2117–2128, 2022.